

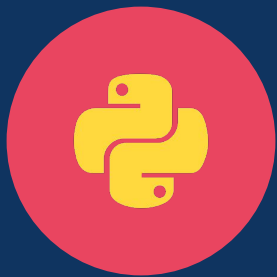
nanoGPT

架构学习

从 Python 基础到 Transformer 全链路

Python · 线性代数 · 注意力机制 · 预训练 · LoRA





模块 一

Python 语法基础

nanoGPT 代码中的必备语法

基础数据结构



list

有序可变序列

```
[1, 2, 3]
```

dict

键值对映射

```
{'a': 1}
```

tuple

有序不可变序列

```
(1, 2, 3)
```

set

无序去重集合

```
{1, 2, 3}
```

nanoGPT 实例: 从文本构建字符表

```
text = "hello"
```

```
set(text)          → {'h', 'e', 'l', 'o'}    # 去重
```

```
list(set(text))    → ['h', 'e', 'l', 'o']    # 转 list
```

```
sorted(...)        → ['e', 'h', 'l', 'o']    # 排序
```

列表推导式与 enumerate



推导式写法

```
chars = ['e','h','l','o']  
  
stoi = {ch: i  
        for i, ch  
        in enumerate(chars)}
```

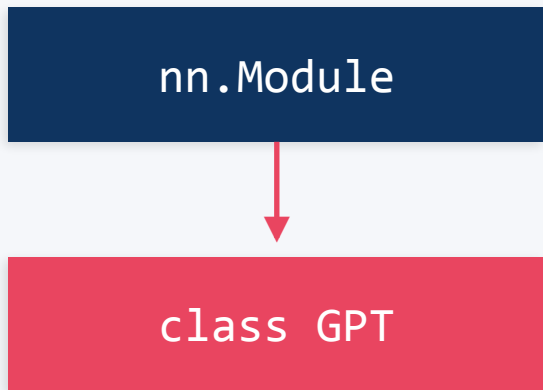
等价 for 循环

```
stoi = {}  
for i, ch in enumerate(chars):  
    stoi[ch] = i
```

输出结果

i	ch	stoi[ch]=i
0	'e'	{'e': 0}
1	'h'	{'e':0, 'h':1}
2	'l'	{'e':0, 'h':1, 'l':2}
3	'o'	{'e':0, 'h':1, 'l':2, 'o':3}

类与继承(nn.Module)



__init__
forward
self

初始化模型组件(层、参数)
定义数据如何流过模型
引用实例本身, 存储属性

nanoGPT 模型骨架

```
class GPT(nn.Module):
    def __init__(self, config):
        super().__init__()
        self.transformer = ...
        self.lm_head = ...

    def forward(self, idx):
        # 前向传播逻辑
        return logits, loss

    def generate(self, idx):
        # 自回归生成
```

装饰器与上下文管理器



训练模式 Training

```
# 梯度开启(默认)
model.train()
loss = model(x, y)
loss.backward() # 计算梯度
optimizer.step() # 更新参数
```

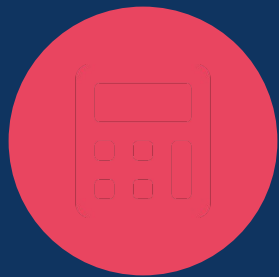
✓ 梯度计算 ✓ Dropout 激活 ✓ 参数更新

推理模式 Inference

```
@torch.no_grad()
def generate(self, idx):
    model.eval()
    with torch.autocast():
        logits = model(idx)
```

✗ 不计算梯度 ✗ Dropout 关闭 → 节省显存

@torch.no_grad() 是装饰器——在函数外面包一层，自动关闭梯度。with 是上下文管理器——在代码块内自动管理资源。

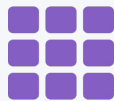


模块二

数学基础

Transformer 背后的核心数学概念

向量与矩阵



向量 Vector

一个 token 的 embedding

0.2
-0.1
0.8

维度 = 3

矩阵 Matrix

一个 batch 的 embeddings

token ₁	0.2	-0.1	0.8
token ₂	0.5	0.3	-0.2
token ₃	-0.4	0.7	0.1

(3×3) 矩阵 = 3 个向量堆叠

Transformer 中, token 用向量表示, 一个句子/ batch 用矩阵表示。所有计算本质上都是矩阵运算。

向量内积(点积)

$$a \cdot b = \sum(a_i \times b_i) \quad \text{对应元素相乘, 再求和} \rightarrow \text{得到一个标量}$$

完整计算示例

a =

1

2

3

b =

0.5

-0.3

0.8

$$\begin{aligned} a \cdot b &= (1 \times 0.5) + (2 \times (-0.3)) + (3 \times 0.8) \\ &= 0.5 + (-0.6) + 2.4 \\ &= 2.3 \end{aligned}$$

在 Attention 中:

Query · Key = 相似度
点积越大 → 越相关

矩阵转置



行列互换: $(m \times n) \rightarrow (n \times m)$

A (2×3)

1	2	3
4	5	6

→ 转置 →

A^T (3×2)

1	4
2	5
3	6

元素位置变化: $A[i][j] \rightarrow A^T[j][i]$

在 nanoGPT 中的用途:

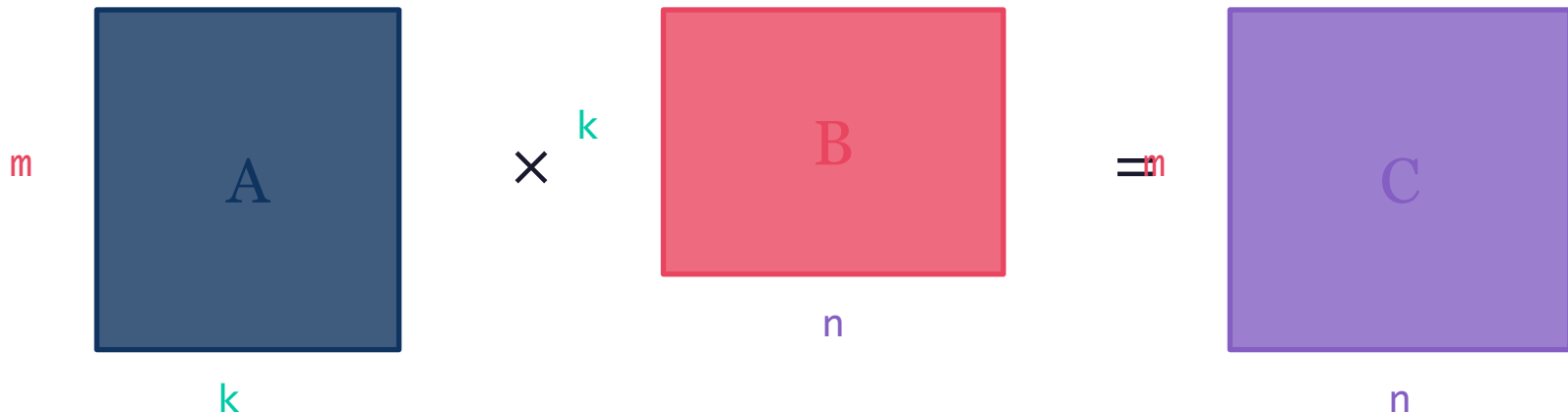
`Q @ K.transpose(-2, -1)`

K 转置后, $Q(T \times d) \times K^T(d \times T) = (T \times T)$ 注意力得分矩阵

矩阵乘法(一) 规则

$$(m \times k) \times (k \times n) \rightarrow (m \times n)$$

中间维度 k 必须相同



$C_{ij} = A \text{ 的第 } i \text{ 行} \cdot B \text{ 的第 } j \text{ 列 (内积)}$

矩阵乘法(二)完整推 导

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

C₁₁ A 行1 [1,2] · B 列1 [5,7]

$$(1 \times 5) + (2 \times 7) = 5 + 14 = \mathbf{19}$$

C₁₂ A 行1 [1,2] · B 列2 [6,8]

$$(1 \times 6) + (2 \times 8) = 6 + 16 = \mathbf{22}$$

C₂₁ A 行2 [3,4] · B 列1 [5,7]

$$(3 \times 5) + (4 \times 7) = 15 + 28 = \mathbf{43}$$

C₂₂ A 行2 [3,4] · B 列2 [6,8]

$$(3 \times 6) + (4 \times 8) = 18 + 32 = \mathbf{50}$$

Softmax

$$\text{softmax}(z_i) = e^{z_i} / \sum e^{z_j}$$

任意实数 $\rightarrow$ 概率分布 (> 0 , 求和 = 1)

完整推导

输入: $z = [2.0, 1.0, 0.1]$

Step 1 指数化:

$$e^{2.0} = 7.39, e^{1.0} = 2.72, e^{0.1} = 1.11$$

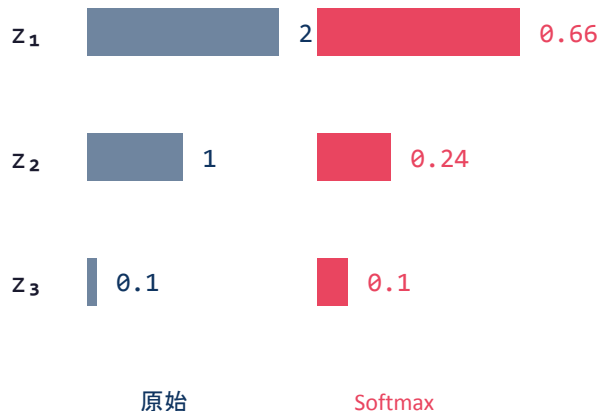
Step 2 求和:

$$7.39 + 2.72 + 1.11 = 11.22$$

Step 3 归一化:

$$[7.39/11.22, 2.72/11.22, 1.11/11.22] \\ = [0.66, 0.24, 0.10]$$

原始分数 vs 概率



交叉熵 Cross Entropy

$$L = -\sum y_i \log(p_i) \quad \text{one-hot 简化为} \quad L = -\log(p_{\text{correct}})$$

预测一般

真实: [0, 0, 1, 0]

预测: [0.1, 0.2, 0.6, 0.1]

$$\text{Loss} = -\ln(0.6) = 0.51$$

预测变好

真实: [0, 0, 1, 0]

预测: [0.05, 0.05, 0.85, 0.05]

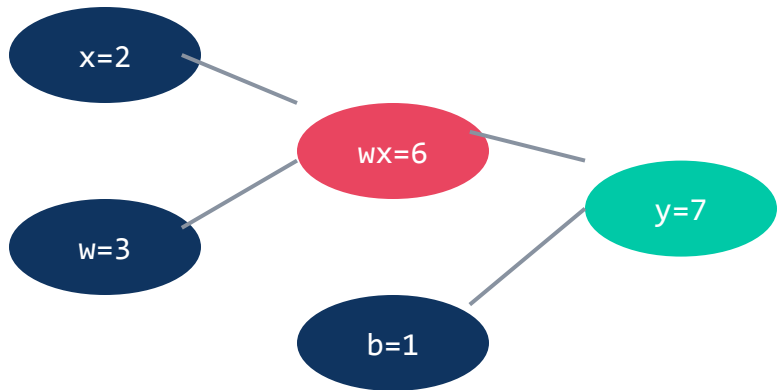
$$\text{Loss} = -\ln(0.85) = 0.16 \quad \downarrow \text{更低!}$$

p_{correct} 越接近 1 $\rightarrow -\ln(p)$ 越接近 0 $\rightarrow$ Loss 越小 $\rightarrow$ 模型越好

梯度与反向传播



计算图: $y = wx + b$



反向传播(链式法则)

假设 $L = (y - \text{target})^2$

$$\partial L / \partial y = 2(y - \text{target})$$

$$\begin{aligned} \partial L / \partial w &= \partial L / \partial y \times \partial y / \partial w \\ &= \partial L / \partial y \times x \end{aligned}$$

$$\begin{aligned} \partial L / \partial b &= \partial L / \partial y \times \partial y / \partial b \\ &= \partial L / \partial y \times 1 \end{aligned}$$

PyTorch 一行搞定:

loss.backward()

核心循环: forward $\rightarrow$ 算 loss $\rightarrow$ backward(算梯度) $\rightarrow$ optimizer.step(更新参数) $\rightarrow$ 重复



模块 三

nanoGPT 核心知识点

从 Tokenizer 到 Transformer 完整拆解

B, T, C — nanoGPT 三大维度

贯穿 nanoGPT 代码的三个核心维度约定

B

Batch Size
= 并发量

一次喂给模型多少条独立文本序列

B=4 → 同时处理 4 段文本

`nanoGPT: batch_size`

类比: 餐厅同时接待 4 桌客人

T

Sequence Length
= 上下文窗口 长度

每条序列包含多少个 token

T=8 → 每条序列看 8 个 token

`nanoGPT: block_size`

类比: 每篇文章限读 8 个字

C

Embedding Dim
= 特征/属性 维度

每个 token 用多少维向量表示

C=64 → 每个 token 是 64 维向量

`nanoGPT: n_embd`

类比: 用 64 种属性描述一个字

数据流维度变化

输入 Token IDs

(B, T)

整数矩阵

Embedding

(B, T, C)

ID → C 维向量

Transformer

(B, T, C)

信息提炼

Output Head

(B, T, V)

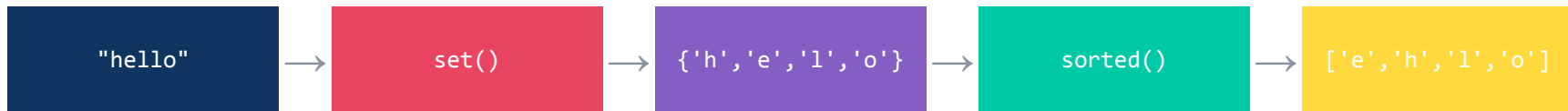
预测分布

核心直觉

B 影响训练速度(并行度), 不影响模型 结构

T 决定上下文窗口大小 | C 决定模型表达能力 | 全流程: (B,T) → (B,T,C) → (B,T,V)

字典构建 (Tokenizer)



双向映射

char → int (stoi)	
'e'	0
'h'	1
'l'	2
'o'	3

编码 "hello"

'h' → 1, 'e' → 0, 'l' → 2, 'l' → 2, 'o' → 3

"hello" → [1, 0, 2, 2, 3]

BPE(子词分词): 实际 GPT 使用 Byte-Pair Encoding, 将高频字符对合并为子词 token。例如 "th" + "e" → "the" 作为一个 token。nanoGPT 教学版用字符级分词简化管理。

BPE 字节对编码

Byte-Pair Encoding — GPT 系列的实际 Tokenizer 方案

BPE 合并过程示例

初始词表: [a, b, c, d, e, r]

语料: a b r a c a d a b r a

第 1 轮 最频繁 pair = (a, b) → 合并为 "ab"

ab r a c a d ab r a

第 2 轮 最频繁 pair = (ab, r) → 合并为 "abr"

abr a c a d abr a

第 3 轮 最频繁 pair = (abr, a) → 合并为 "abra"

abra c a d abra

第 4 轮 最频繁 pair = (a, d) → 合并为 "ad"

abra c ad abra

最终词表: [a, b, c, d, e, r, ab, abr, abra, ad]

词表从 6 → 10, 高频组合获得独立 token

Byte-level BPE (GPT 实际方案)

基础词表 = 256 个 UTF-8 字节

在此基础上做 BPE 合并

优势:

- 任何语言都能处理(中/日/韩/emoji)
- 不会出现 unknown token
- GPT-2 词表大小: 50,257

三种分词方式对比

字符级	nanoGPT 教学版	'h','e','l','l', 'o'
子词级 (BPE)	GPT-2/3/4 实际	'he'l','l'o'
字节级 BPE	GPT 最终方案	UTF-8字节→合并

与 nanoGPT 的关系

nanoGPT 教学版用字符级分词简化管理 · 实际 GPT-2/3/4 使用 Byte-level BPE · 原理相同, 粒度不同

Embedding (词嵌入)



`nn.Embedding(vocab_size, n_embd)` – 本质是一个查找表

Embedding 查找表 (vocab=4, dim=3)

ID 0 ('e')	0.12	-0.34	0.56
ID 1 ('h')	0.78	0.23	-0.11
ID 2 ('l')	-0.45	0.67	0.33
ID 3 ('o')	0.91	-0.08	0.44

← token ID=1 查表

得到 [0.78, 0.23, -0.11]

为什么需要 Embedding?

整数 ID 没有语义信息

向量可以表达相似性

"猫"和"狗"的向量距离近

位置编码 Positional Encoding

Transformer 没有内置的顺序感知 → 需要位置编码告诉模型 "这是第几个 token"

Token Embedding + Position Embedding = 最终输入

Token Emb	0.78	0.23	-0.11	+			
Position Emb	0.10	-0.05	0.20	=	最终输入		
					0.88	0.18	0.09

```
self.wpe = nn.Embedding(block_size, n_embd) # 可学习的位置编码
```

文本: "abcdef" block_size=4

X = a b c d Y = b c d e

训练时逐步扩展上下文:

输入 [a]	→	预测 b
输入 [a, b]	→	预测 c
输入 [a, b, c]	→	预测 d
输入 [a, b, c, d]	→	预测 e

$$X =$$

a

b

C

d

$$Y =$$

b

C

d

e

输入 [a]

→ 预测 b

输入 $[a, b]$

→ **预测 C**

输入 [a, b, c]

→ 预测 d

输入 [a, b, c, d]

→ 预测 e

QKV 构建



$(T \times d)$

Q

Query

$X \times W_q$

我在找什么？

K

Key

$X \times W_k$

我有什么标签？

V

Value

$X \times W_v$

我的信息内容

图书馆类比

Q = 你的搜索词 "我想找关于机器学习的书"

K = 每本书的标签 "AI""深度学习""烹饪""历史"...

V = 书的内容 Q-K 匹配度高的书 → 取出对应 V(内容)

自注意力机制(一)公式

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) \times V$$

Step 1

$$Q \cdot K^T$$

算出每对 token 的相关性得分

维度: $(T \times d) \times (d \times T) = (T \times T)$

Step 2

$$\div \sqrt{d}$$

缩放防止数值过大
避免 softmax 梯度消失

Step 3

Softmax

归一化为概率分布
每行和 = 1

Step 4

$$\times V$$

用概率加权汇总信息
 $(T \times T) \times (T \times d) = (T \times d)$

维度流: $Q(T \times d) \times K^T(d \times T) \rightarrow \text{得分}(T \times T) \rightarrow \text{softmax}(T \times T) \rightarrow \times V(T \times d) \rightarrow \text{输出}(T \times d)$

自注意力机制(二)数值示例

3 个 token, d=2 的小例子

Q	1	0	K	1	1	V	0.5	0.3	$Q \cdot K^T / \sqrt{2} =$	0.71	0.00	0.71
	0	1		0	1		0.2	0.8		0.71	0.71	0.00
	1	1		1	0		0.9	0.1		1.41	0.71	0.71

Softmax 后的注意力 权重(热力图)

	t_1	t_2	t_3
t_1	0.39	0.22	0.39
t_2	0.36	0.36	0.28
t_3	0.40	0.30	0.30

最终输出 = Weights $\times$ V

$$\begin{aligned} t_1: & 0.39 \times [0.5, 0.3] + 0.22 \times [0.2, 0.8] \\ & + 0.39 \times [0.9, 0.1] \\ & = [0.59, 0.33] \end{aligned}$$

每个 token 的输出 = 对所有 V 的
加权平均, 权重由 Q·K 决定

因果遮罩 Causal Mask

GPT 是生成模型: 预测时只能看到前面的 token, 不能偷看后面

遮罩前(原始分数)

2.1	$-\infty$	$-\infty$	$-\infty$
1.1	1.8	$-\infty$	$-\infty$
0.9	1.2	2.0	$-\infty$
0.6	0.3	1.1	1.9



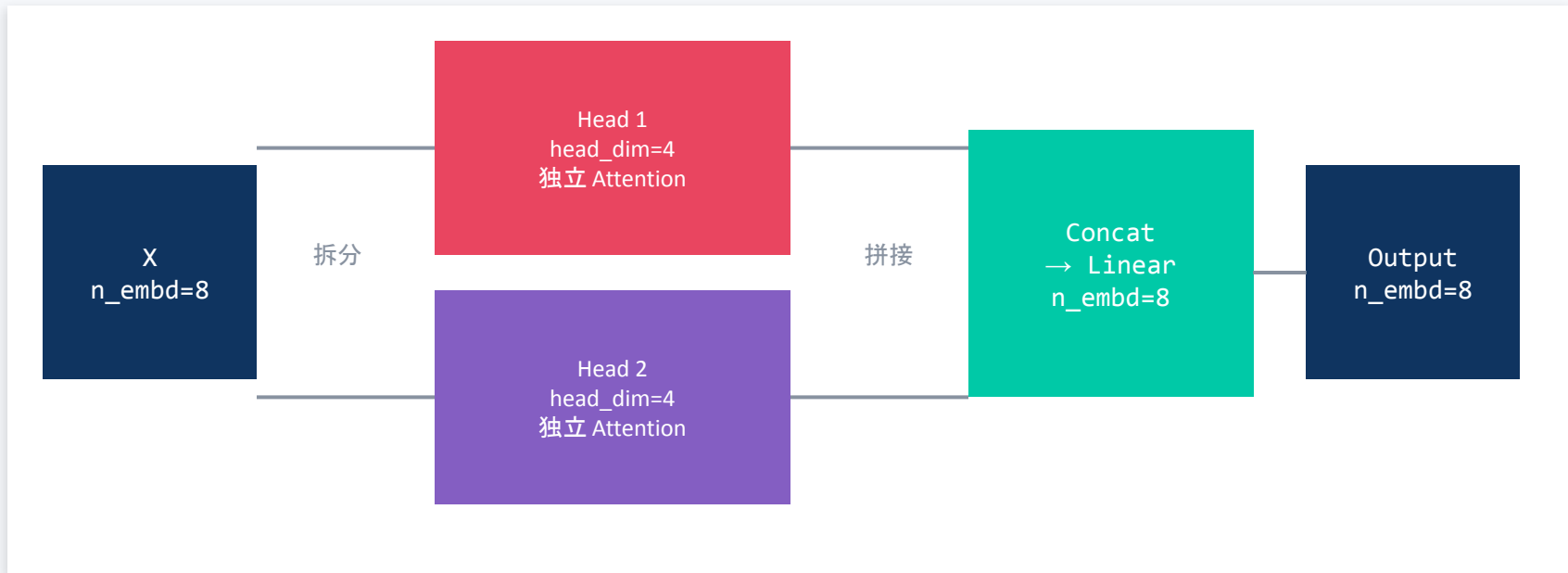
Softmax 后(注意力权重)

1.00	0.00	0.00	0.00
0.33	0.67	0.00	0.00
0.18	0.25	0.57	0.00
0.12	0.09	0.20	0.59

```
tril = torch.tril(torch.ones(T, T)) # 下三角矩阵
```

多头注意力 Multi-Head Attention

把 QKV 拆成多个"头", 每个头独立做 Attention, 最后拼接



$n_head=2$, $head_dim = n_embd/n_head = 8/2 = 4$. 多头 = 多视角, 不同头关注不同语义模式

前馈网络 MLP / Feed-Forward

输入 n_embd

Linear $\rightarrow$

$4 \times n_embd$ (扩维)

GELU 激活 $\rightarrow$

输出 n_embd (压回)

nanoGPT 代码

```
self.mlp = nn.Sequential(  
    nn.Linear(n_embd,  
              4*n_embd),  
    nn.GELU(),  
    nn.Linear(4*n_embd,  
              n_embd),  
)
```

Attention 收集信息

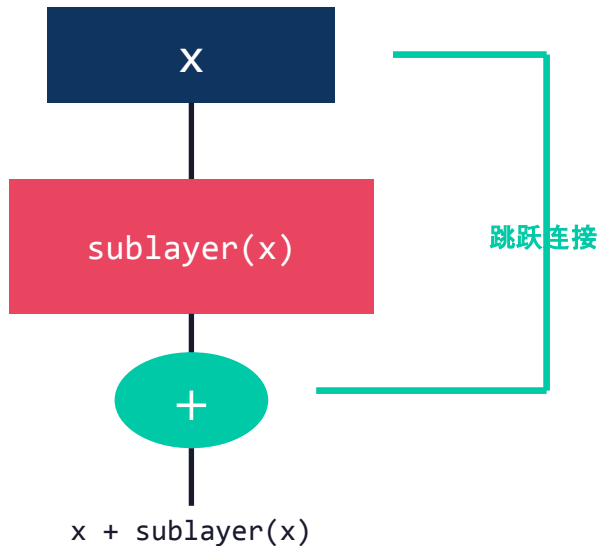
MLP 消化理解

扩维 $4\times$ 给模型更大的特征空间做非线性变换，再压回原维度

残差连接 Residual Connection

$$x = x + \text{sublayer}(x)$$

输出 = 原始输入 + 子层输出



为什么需要残差连接？

1. 防止梯度消失

梯度可以直接通过跳跃连接回传到浅层

2. 保底机制

即使 sublayer 学不好
原始 x 的信息不会丢失

3. 支持深层网络

nanoGPT 6-12层 Block
没有残差连接几乎无法训练

LayerNorm (层归一化)

$$\text{LayerNorm}(x) = \gamma \times (x - \mu) / \sigma + \beta \quad \mu=\text{均值}, \sigma=\text{标准差}, \gamma/\beta \text{ 可学习}$$

完整计算示例

输入向量: $x = [4, 2, 6]$

Step 1 均值: $\mu = (4+2+6)/3 = 4.0$

Step 2 标准差: $\sigma = \sqrt{((4-4)^2 + (2-4)^2 + (6-4)^2)/3}$
 $= \sqrt{(8/3)} = 1.63$

Step 3 归一化:

$$(4-4)/1.63 = 0.00$$

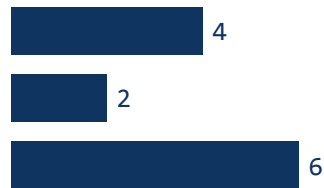
$$(2-4)/1.63 = -1.22$$

$$(6-4)/1.63 = 1.22$$

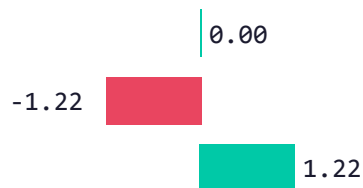
输出: $[0.00, -1.22, 1.22]$

归一化前后对比

归一化前



归一化后



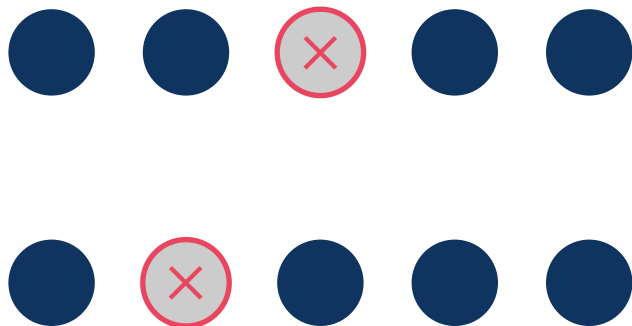
nanoGPT 使用 Pre-Norm: 先 LayerNorm → 再进 Attention/MLP → 最后残差

Dropout

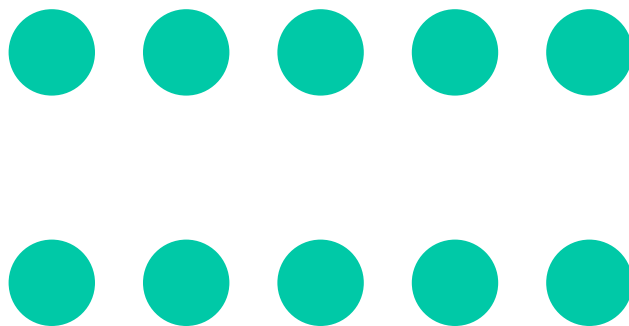


训练时随机"关闭"一部分神经元(设为 0), 推理时全部激活

训练时 ($p=0.2$)



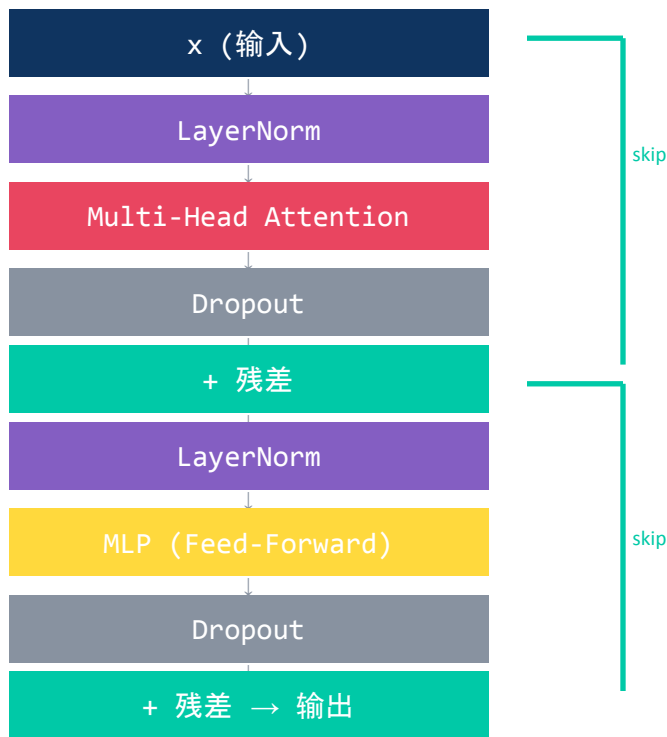
推理时 (全部激活)



nanoGPT 中的位置: Attention 权重后 + MLP 输出后

```
self.dropout = nn.Dropout(dropout)
```

Transformer Block 全景



一个 Block 的公式

```
# Attention 分支  
 $x = x + \text{attn}(\text{ln1}(x))$ 
```

```
# MLP 分支  
 $x = x + \text{mlp}(\text{ln2}(x))$ 
```

nanoGPT 堆叠

n_{layer} 个 Block
通常 6 ~ 12 层

每一层都在提炼更
高层次的语义特征

GPT 参数量推导

参数量逐项拆解

① Token Embedding $V \times C$

② Position Embedding $T \times C$

③ 每个 Transformer Block ($\times N$)

└ LayerNorm $\times 2$	$4C$	// γ, β 各 C
└ QKV 投影	$3C^2 + 3C$	// Q, K, V 投影
└ 输出投影	$C^2 + C$	// 注意力输出
└ MLP fc1	$4C^2 + 4C$	// 扩维 $\times 4$
└ MLP fc2	$4C^2 + C$	// 降回原维

单个 Block 合计 $\approx 12C^2 + 13C$

④ N 个 Block $N \times (12C^2 + 13C)$

⑤ 最终 LayerNorm $2C$

⑥ 输出 Head $C \times V$ (与①共享权重)

总参数量 $\approx V \cdot C + T \cdot C + N \cdot (12C^2 + 13C) + 2C$

核心直觉

• 参数量由 $N \times 12C^2$ 主导

参数量公式直觉

$N \times 12C^2$ 主导 · C 翻倍参数量翻 4 倍 · nanoGPT 10.8M $\rightarrow$ GPT-2 124M $\rightarrow$ GPT-3 175B

nanoGPT Small 数值计算

$V=65$ $C=384$ $N=6$ $T=256$ $h=6$

Token Emb	65×384	24,960
Position Emb	256×384	98,304
每个 Block	$12 \times 384^2 + 13 \times 384$	1,774,464
6 个 Block	$6 \times 1,774,464$	10,646,784
最终 LN		768

$\approx 10,770,816 \approx 10.8M$ 参数

规模对比

nanoGPT Small	10.8M	$C=384, N=6$
GPT-2 Small	124M	$C=768, N=12$
GPT-2 Large	774M	$C=1280, N=36$
GPT-3	175B	$C=12288, N=96$

GPT 完整前向传播

从输入 token IDs 到输出 logits 的端到端数据流

输入	token IDs (B, T)	整数序列
Token Emb + Pos Emb	$(B, T) \rightarrow (B, T, C)$	查表+位置编码
Transformer Block $\times N$	$(B, T, C) \rightarrow (B, T, C)$	N 层注意力+MLP
Final LayerNorm	$(B, T, C) \rightarrow (B, T, C)$	最终归一化
Linear Head	$(B, T, C) \rightarrow (B, T, V)$	投影到词表
输出 logits	$(B, T, \text{vocab_size})$	预测分布

nanoGPT forward 函数

GPT.forward(self, idx, targets=None)

```
# idx: (B, T) token IDs
B, T = idx.size()

# 1. Embedding
tok_emb = self.wte(idx)      # (B,T,C)
pos_emb = self.wpe(pos)      # (T,C)
x = tok_emb + pos_emb        # (B,T,C)

# 2. N 个 Transformer Block
for block in self.blocks:
    x = block(x)              # (B,T,C)

# 3. Final LayerNorm
x = self.ln_f(x)              # (B,T,C)

# 4. Linear Head → logits
logits = self.lm_head(x)      # (B,T,V)
```

架构 → 训练的桥梁

forward 产生 logits → 与 targets 算 cross_entropy loss → backward 更新参数 → 下一页详解

Loss 构建



```
loss = F.cross_entropy(logits.view(-1, vocab_size), targets.view(-1))
```

logits reshape 过程

```
logits: (B, T, vocab_size)
      ↓ view(-1, vocab_size)
logits: (B*T, vocab_size)

targets: (B, T)
      ↓ view(-1)
targets: (B*T,)
```

训练循环

```
for step in range(max_iters):
    xb, yb = get_batch()
    logits, loss = model(xb, yb)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Forward
前向传播



Loss
计算损失



Backward
反向传播



Step
更新参数

预训练 Pre-training



在大规模无标注文本上，通过"预测下一个 token"来学习语言规律

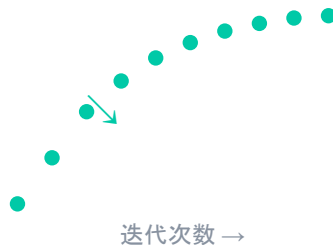
训练数据

- 维基百科
- 新闻文章
- 书籍 / 论文
- 代码 / 网页

关键超参数

```
learning_rate: 3e-4  
batch_size: 64  
warmup_steps: 200  
max_iters: 5000  
eval_interval: 500
```

Loss 曲线



预训练的目标: 让模型通过海量文本学会语言的统计规律、语法结构、世界知识。
预训练好的模型 = 通用语言能力基座，可以用于各种下游任务。

微调与 LoRA



全量微调 Full Fine-tuning



需要大量 GPU 显存

LoRA (低秩适配)



仅训练 $A \times B$, 参数量 $\ll W$

	全量微调	LoRA
训练参数	100%	0.1% ~ 1%
显存需求	极高	低
训练速度	慢	快
效果	最优	接近最优

推理与生成 Inference



自回归生成过程

Step 1: "今天天气"

Step 2: "今天天气真"

Step 3: "今天天气真好"

Step 4: "今天天气真好啊"

→ "真"

→ "好"

→ "啊"

→ "!"

循环直到 <EOS> 或达到最大长度

Temperature 温度

T=0.1 (低温 / 保守):

概率集中, 总是选最可能的词

T=1.0 (标准):

按原始概率采样

T=2.0 (高温 / 创意):

概率更平均, 更多随机性

Top-k & Top-p 采样

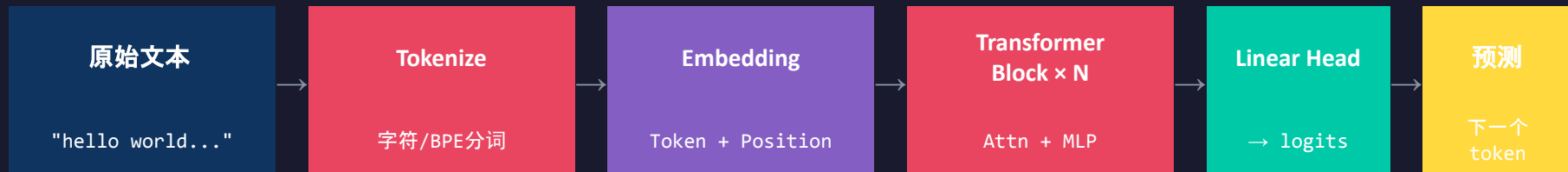
Top-k = 50:

只从概率最高的 50 个词中采样
过滤掉不合理的低概率词

Top-p = 0.9 (nucleus):

从累积概率达 90% 的词中采样
动态调整候选数量

总结:nanoGPT 端到端全景



模块一: Python 基础

数据结构 · 推导式
类与继承 · 装饰器



模块二: 数学基础

向量/矩阵 · 内积 · 转置
Softmax · 交叉熵 · 梯度



模块三: nanoGPT 核心

Tokenizer · Embedding · QKV
Attention · MLP · LayerNorm
预训练 · LoRA · 推理生成

理解 nanoGPT = 理解 GPT 的核心原理。掌握这些知识, 你就拥有了读懂任何 Transformer 模型的能力。